

Microsoft Fabric Warehouse

Database Administration Complete Learning Material

Based on Official Microsoft Documentation
With Architecture Diagrams & DBA SQL Query Library
Source: learn.microsoft.com/en-us/fabric

15 Topics	80+ Hours	60+ Diagrams	100+ SQL Queries	Official MS Docs Links
-----------	-----------	--------------	------------------	------------------------

April 2025 | Version 1.0 | English Language Edition

How to Use This Learning Material

This learning material is structured as a complete curriculum for Microsoft Fabric Warehouse Database Administration. It is based entirely on official Microsoft documentation from learn.microsoft.com/fabric. Each topic includes:

- Architecture Diagrams — visual representations of concepts and data flows
- Explanatory Content — clear descriptions based on Microsoft official docs
- DBA SQL Queries — ready-to-use T-SQL scripts for each administration task
- Official Reference Links — direct URLs to the Microsoft documentation pages

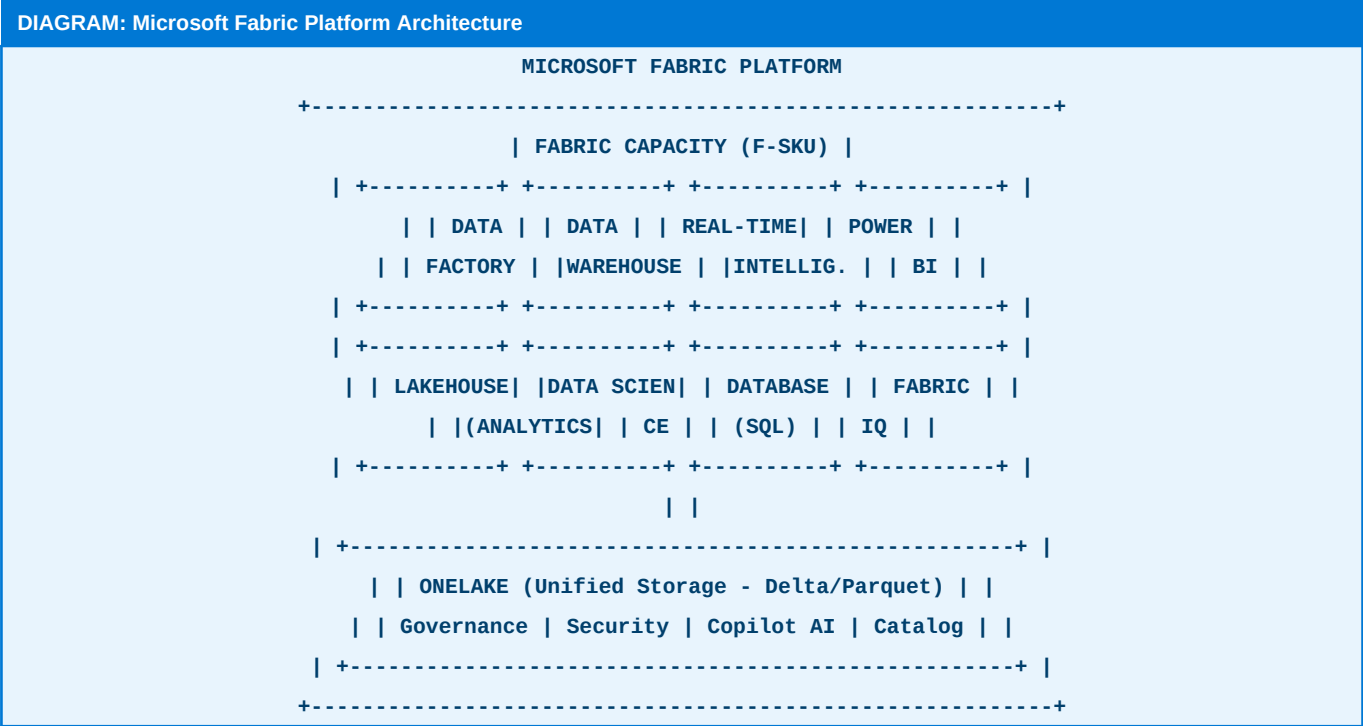
DIAGRAM	SQL QUERY	NOTE / TIP	MS DOCS REF
---------	-----------	------------	-------------

Topic Index

01	Introduction to Microsoft Fabric	Platform overview, OneLake, Fabric components, DBA context
02	Microsoft Fabric Licensing & Cost	CU model, F-SKU tiers, cost optimization, capacity management
03	Workspace in Microsoft Fabric & Power BI	Workspace roles, deployment pipelines, Git, sensitivity labels
04	Fabric Warehouse Fundamentals	Architecture, table types, data types, stored procedures, views
05	Warehouse Security	Auth, GRANT/DENY, RLS, column security, DDM, audit
06	Fabric Warehouse Capacity & Monitoring	DMVs, Query Insights, live monitoring, KILL sessions
07	Data Loading & Ingestion	COPY INTO, staging patterns, MERGE, incremental loads
08	Transactions, Locking & Concurrency	Snapshot isolation, write-write conflicts, retry patterns
09	Performance Tuning & Optimization	Statistics, query design, compaction, anti-patterns
10	Disaster Recovery & Business Continuity	BCDR, restore-in-place, cross-region DR, recovery scripts
11	Source Control, CI/CD & DevOps	Git integration, idempotent scripts, deployment automation
12	Database Mirroring & Real-Time Sync	CDC setup, mirror monitoring, cross-database queries
13	Integration with External Services & Tools	Power BI, Lakehouse, SSMS, Python, third-party tools
14	Fabric Administration & Governance	Admin portal, audit, Purview, compliance, object inventory
15	DBA Master Reference & SQL Query Library	Daily health check, weekly reports, troubleshooting scripts

01 Introduction to Microsoft Fabric

Microsoft Fabric is a unified, end-to-end analytics and data platform delivered as a Software-as-a-Service (SaaS) solution on Azure. It brings together data engineering, data integration, data warehousing, real-time analytics, data science, and business intelligence into a single, cohesive platform — all sharing one storage layer called OneLake.

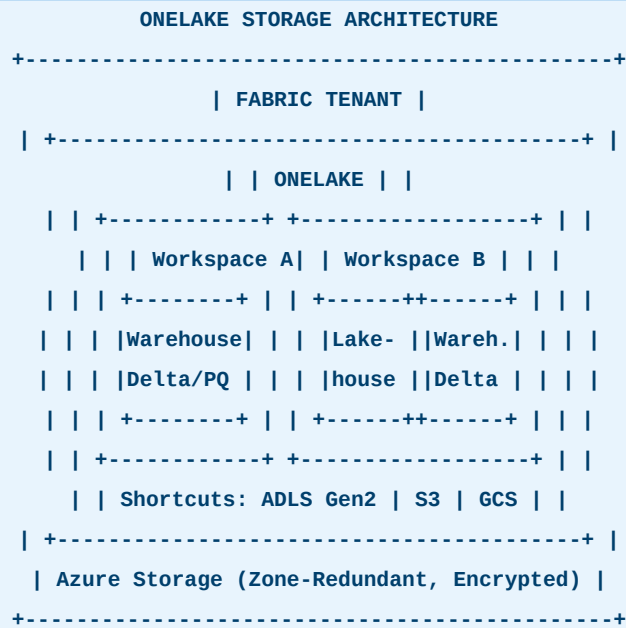


1.1 Fabric Components Explained

Component	Workload Category	DBA Relevance
Data Factory	Data Integration	Pipeline orchestration, ETL into Warehouse
Data Warehouse	Analytics	Primary DBA focus — SQL-based analytics engine
Lakehouse	Analytics	Cross-database queries from Warehouse
Real-Time Intelligence	Analytics	Streaming data landing into Warehouse
Data Science	Analytics	ML models reading Warehouse data
Power BI	Business Intelligence	Reporting on Warehouse data
SQL Database	Databases	OLTP workloads, different from Warehouse
OneLake	Foundation	Unified Delta Lake storage for all workloads
Fabric IQ	AI/Governance	Semantic layer and AI assistant (Copilot)

1.2 OneLake Architecture

OneLake is the single unified storage layer for all Microsoft Fabric workloads. Built on Azure Data Lake Storage Gen2, it uses Delta Lake format (Parquet files with transaction logs) as the standard open data format. All Warehouses, Lakehouses, and SQL Databases store data in OneLake automatically.

DIAGRAM: OneLake Storage Hierarchy**1.3 DBA SQL Queries — Fabric Platform Exploration****SQL: List All Schemas in Fabric Warehouse**

```
-- View all schemas in your Warehouse
SELECT schema_id, name AS schema_name,
SCHEMA_OWNER = USER_NAME(principal_id)
FROM sys.schemas
ORDER BY name;
```

SQL: List All Tables with Row Counts

```
-- Inventory of all user tables in the Warehouse
SELECT
s.name AS schema_name,
t.name AS table_name,
t.create_date AS created_date,
t.modify_date AS last_modified
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE t.is_ms_shipped = 0
ORDER BY s.name, t.name;
```

SQL: Check Fabric Version and Edition

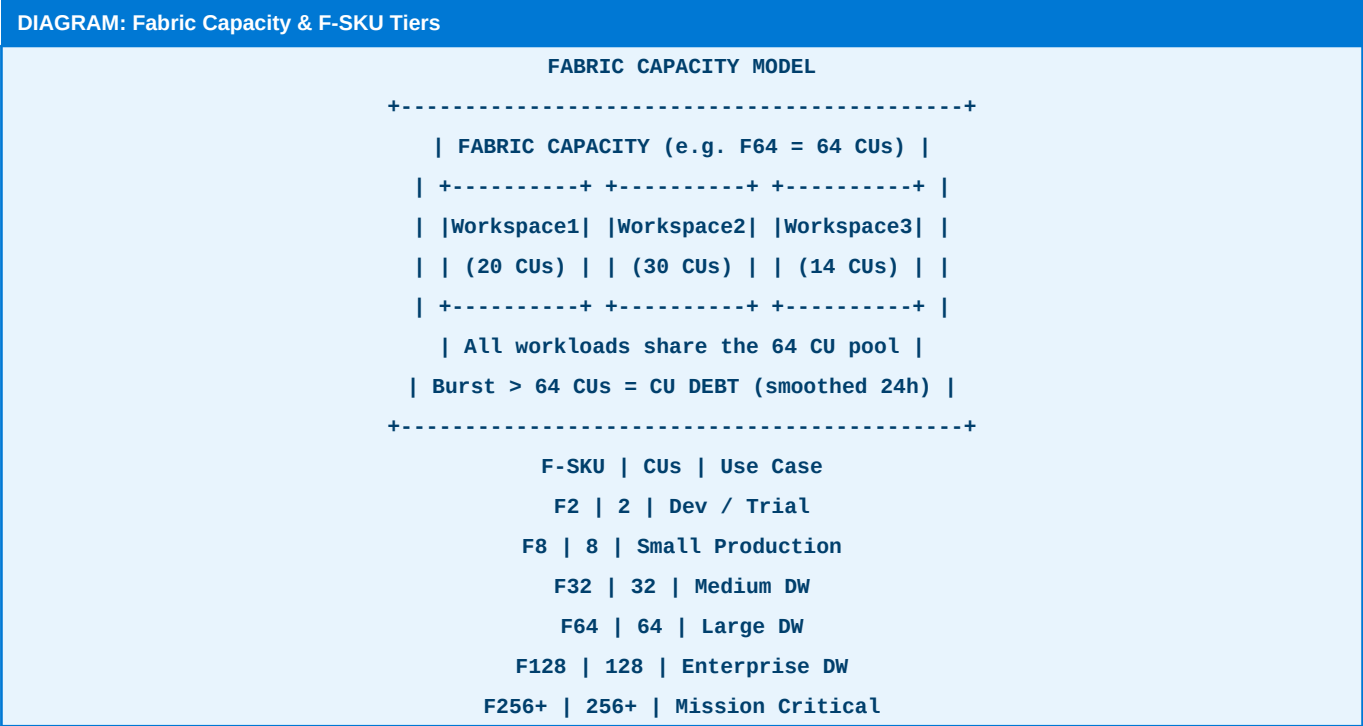
```
-- Confirm Fabric Warehouse engine details
SELECT @@VERSION AS fabric_version,
@@SERVERNAME AS server_name,
DB_NAME() AS current_database,
GETUTCDATE() AS current_utc_time;
```

Official Microsoft Documentation References:

- *What is Microsoft Fabric:* <https://learn.microsoft.com/en-us/fabric/fundamentals/microsoft-fabric-overview>
- *What is Fabric Data Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/data-warehousing>
- *OneLake Overview:* <https://learn.microsoft.com/en-us/fabric/onelake/onelake-overview>
- *Fabric Components:* <https://learn.microsoft.com/en-us/fabric/fundamentals/fabric-terminology>

02 Microsoft Fabric Licensing & Cost

Microsoft Fabric uses a capacity-based licensing model. A Fabric Capacity is a pool of compute resources measured in Capacity Units (CUs). All workloads in workspaces assigned to that capacity share those CUs. The capacity SKU determines the maximum CUs available and the price per hour.



2.1 CU Consumption by Operation

Operation	CU Pattern	DBA Action
SELECT queries	CUs during execution only	Monitor via Capacity Metrics App
INSERT / COPY INTO	CUs during write + compaction	Schedule large loads off-peak
UPDATE / DELETE	Higher CU — read+write	Minimize concurrent DML
Statistics update	Background CU cost	Run after major data loads
Pipeline activities	Per-activity CU charge	Batch small activities
Power BI refresh	CUs per refresh	Stagger report refresh schedules

2.2 DBA SQL Queries — Capacity Monitoring

SQL: Query Duration Distribution for Cost Analysis

```
-- Classify queries by duration bucket for cost profiling
SELECT
CASE
WHEN total_elapsed_time_ms < 1000 THEN 'Under 1 second'
WHEN total_elapsed_time_ms < 5000 THEN '1-5 seconds'
WHEN total_elapsed_time_ms < 30000 THEN '5-30 seconds'
WHEN total_elapsed_time_ms < 60000 THEN '30-60 seconds'
ELSE 'Over 1 minute'
END AS duration_bucket,
COUNT(*) AS query_count,
AVG(total_elapsed_time_ms)/1000.0 AS avg_sec,
SUM(row_count) AS total_rows
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY,-7,GETUTCDATE())
AND status = 'Succeeded'
GROUP BY
CASE
WHEN total_elapsed_time_ms < 1000 THEN 'Under 1 second'
WHEN total_elapsed_time_ms < 5000 THEN '1-5 seconds'
WHEN total_elapsed_time_ms < 30000 THEN '5-30 seconds'
WHEN total_elapsed_time_ms < 60000 THEN '30-60 seconds'
ELSE 'Over 1 minute'
END
ORDER BY MIN(total_elapsed_time_ms);
```

SQL: Top Users by Query Volume and Cost Impact

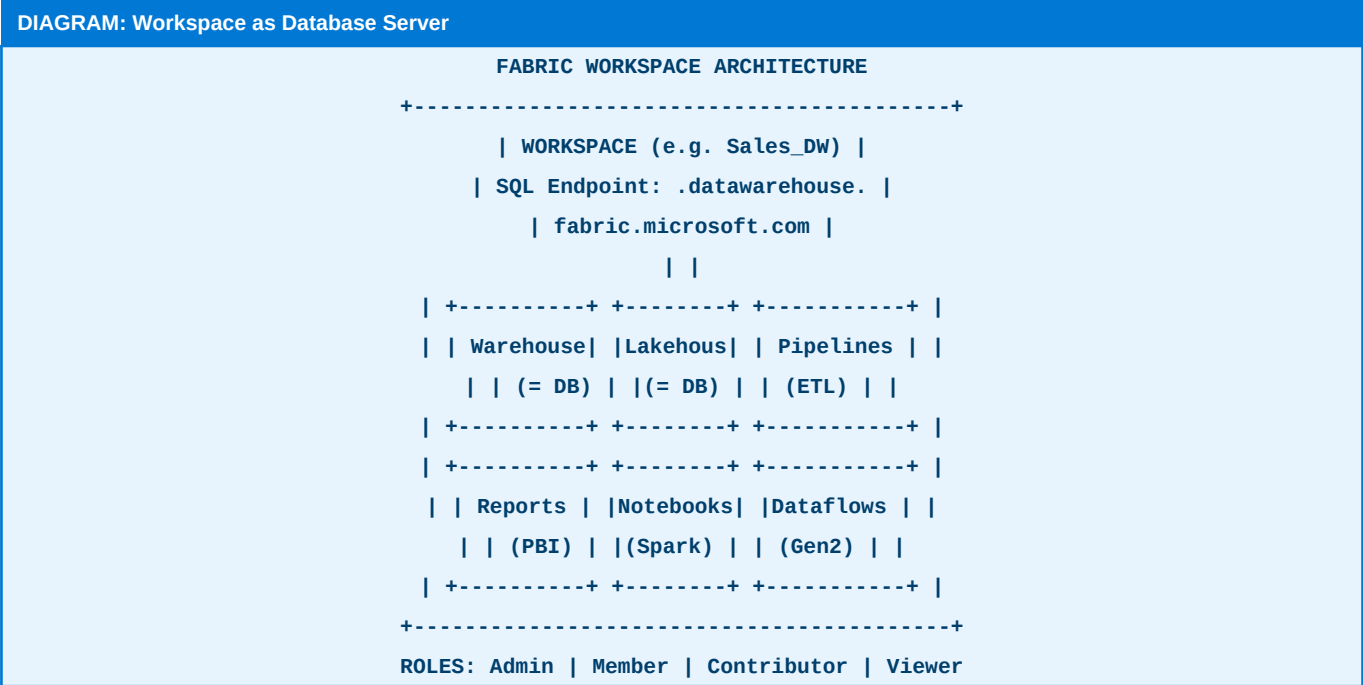
```
SELECT
login_name,
COUNT(*) AS total_queries,
AVG(total_elapsed_time_ms)/1000.0 AS avg_elapsed_sec,
MAX(total_elapsed_time_ms)/1000.0 AS max_elapsed_sec,
SUM(total_elapsed_time_ms)/1000.0 AS total_elapsed_sec,
SUM(row_count) AS total_rows_returned
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY,-30,GETUTCDATE())
GROUP BY login_name
ORDER BY total_elapsed_sec DESC;
```

Official Microsoft Documentation References:

- *Fabric Capacity & Billing*: <https://learn.microsoft.com/en-us/fabric/enterprise/licenses>
- *Capacity Metrics App*: <https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app>
- *Burstable Capacity*: <https://learn.microsoft.com/en-us/fabric/enterprise/burstable-capacity>
- *Cost Optimization*: <https://learn.microsoft.com/en-us/fabric/enterprise/optimize-capacity>

03 Workspace in Microsoft Fabric & Power BI

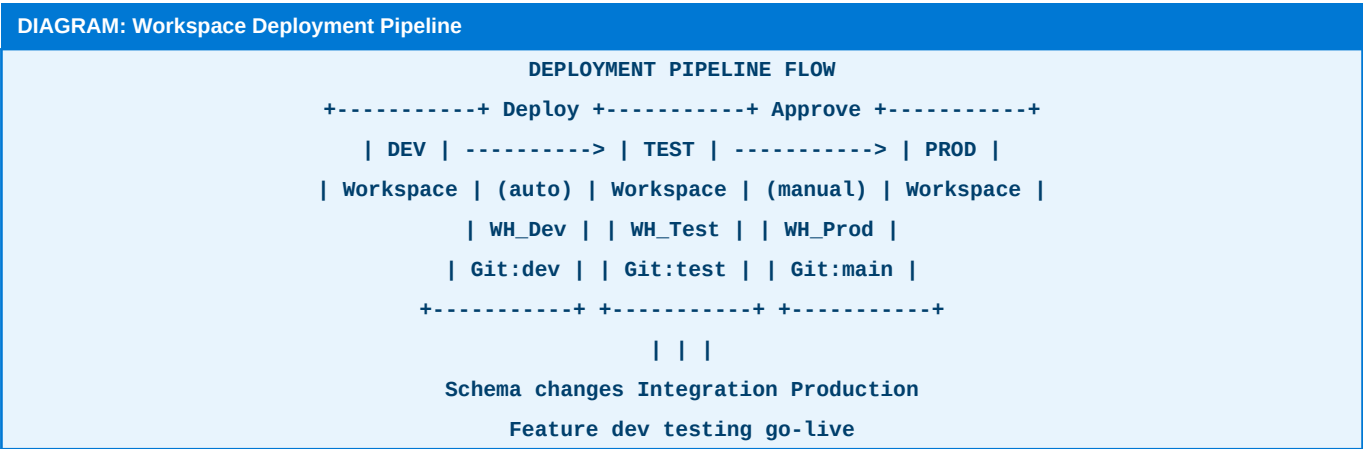
A Fabric Workspace is the primary organizational container for all Fabric items including Warehouses, Lakehouses, Pipelines, Notebooks, and Reports. From a DBA perspective, the workspace acts like a SQL Server instance — when connecting via SSMS, the workspace SQL endpoint is the server and each Warehouse is a database.



3.1 Workspace Roles & Permissions

Role	Create Items	Read Data	Write Data	Admin Actions	KILL Sessions
Admin	Yes	Yes	Yes	Yes	Yes
Member	Yes	Yes	Yes	Yes	No
Contributor	Yes	Yes	Yes	No	No
Viewer	No	Yes	No	No	No

3.2 Deployment Pipelines — Dev > Test > Prod



3.3 DBA SQL Queries — Workspace & Object Inventory

SQL: List All Views in the Warehouse

```
SELECT
s.name AS schema_name,
v.name AS view_name,
v.create_date,
v.modify_date,
m.definition AS view_definition
FROM sys.views v
JOIN sys.schemas s ON v.schema_id = s.schema_id
JOIN sys.sql_modules m ON v.object_id = m.object_id
ORDER BY s.name, v.name;
```

SQL: List All Stored Procedures

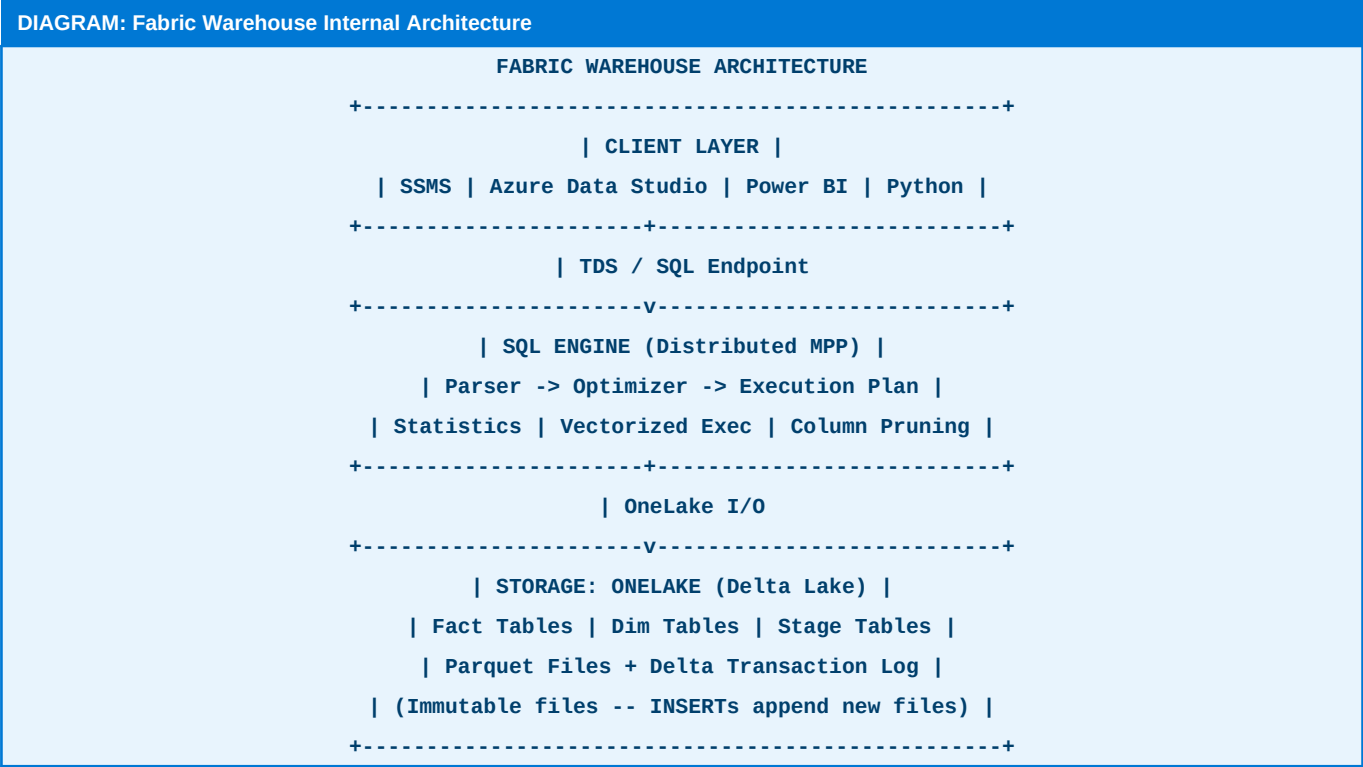
```
SELECT
s.name AS schema_name,
p.name AS procedure_name,
p.create_date,
p.modify_date
FROM sys.procedures p
JOIN sys.schemas s ON p.schema_id = s.schema_id
ORDER BY s.name, p.name;
```

Official Microsoft Documentation References:

- *Workspace Roles*: <https://learn.microsoft.com/en-us/fabric/fundamentals/roles-workspaces>
- *Deployment Pipelines*: <https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines>
- *Sensitivity Labels*: <https://learn.microsoft.com/en-us/fabric/governance/apply-sensitivity-labels>
- *Git Integration*: <https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration>

04 Fabric Warehouse Fundamentals

Fabric Data Warehouse is a lake warehouse — an enterprise-scale relational SQL warehouse built on a Delta Lake foundation. Unlike traditional warehouses, compute and storage are fully separated. Data is stored as Parquet files in OneLake with a Delta transaction log ensuring ACID compliance.



4.1 Table Types & Design

Table Type	Purpose	Naming Convention	Key Characteristics
Fact	Transactional measures	FactSales, FactOrders	Large, narrow, numeric keys+measures
Dimension	Descriptive attributes	DimCustomer, DimDate	Smaller, wider, surrogate keys
Integration/Stage	ETL intermediate	StgOrders, IntOrders	Temporary — cleared after load
#Temp Tables	Session-scoped work	#TempCalc	Dropped on session end
Bridge/Junction	Many-to-many resolution	BridgeProductTag	Links two dimension tables

4.2 Supported Data Types

Category	Data Types	DBA Notes
Integer	TINYINT, SMALLINT, INT, BIGINT	Use INT/BIGINT for surrogate keys
Decimal	DECIMAL(p,s), NUMERIC(p,s)	Use for financial amounts — never FLOAT
Floating	FLOAT, REAL	Avoid for money — rounding errors
String	VARCHAR(n), NVARCHAR(n), CHAR	VARCHAR for ASCII, NVARCHAR for Unicode
Date/Time	DATE, TIME, DATETIME2, DATETIMEOFFSET	Use DATE for date-only columns
Boolean	BIT	Use for flag/boolean columns
Binary	VARBINARY(n)	Limited support — avoid in Warehouse
Unicode Max	NVARCHAR(MAX)	Use sparingly — performance impact

4.3 DBA SQL Queries — Warehouse Fundamentals

SQL: Create a Schema for Organizing Objects

```
-- Create separate schemas for business domains
CREATE SCHEMA sales;
CREATE SCHEMA hr;
CREATE SCHEMA finance;
GO
```

SQL: Create a Fact Table with Constraints

```
CREATE TABLE dbo.FactSales (
    SalesKey BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    CustomerKey INT NOT NULL,
    ProductKey INT NOT NULL,
    TerritoryKey INT NOT NULL,
    SalesAmount DECIMAL(18,2) NOT NULL,
    TaxAmount DECIMAL(18,2) NOT NULL DEFAULT 0,
    Quantity INT NOT NULL DEFAULT 1,
    LoadDate DATE NOT NULL DEFAULT CAST(GETUTCDATE() AS DATE),
    CONSTRAINT PK_FactSales PRIMARY KEY (SalesKey)
);
```

SQL: Create a Dimension Table

```
CREATE TABLE dbo.DimCustomer (
    CustomerKey INT NOT NULL,
    CustomerID NVARCHAR(20) NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(200) NULL,
    Region NVARCHAR(50) NOT NULL,
    Country NVARCHAR(50) NOT NULL,
    Segment NVARCHAR(30) NOT NULL,
    IsActive BIT NOT NULL DEFAULT 1,
    EffectiveDate DATE NOT NULL,
    ExpiryDate DATE NULL,
    CONSTRAINT PK_DimCustomer PRIMARY KEY (CustomerKey)
);
```

SQL: Create a View for Reporting

```
CREATE OR ALTER VIEW dbo.vw_SalesReport AS
SELECT
    f.OrderDate,
    YEAR(f.OrderDate) AS OrderYear,
    MONTH(f.OrderDate) AS OrderMonth,
    c.CustomerName,
    c.Region,
    c.Country,
    c.Segment,
    p.ProductName,
    p.Category,
    f.SalesAmount,
    f.TaxAmount,
    f.Quantity,
    f.SalesAmount + f.TaxAmount AS TotalAmount
FROM dbo.FactSales f
JOIN dbo.DimCustomer c ON f.CustomerKey = c.CustomerKey
JOIN dbo.DimProduct p ON f.ProductKey = p.ProductKey
JOIN dbo.DimTerritory t ON f.TerritoryKey = t.TerritoryKey;
```

SQL: Create a Stored Procedure for ETL Load

```
CREATE OR ALTER PROCEDURE dbo.usp_LoadDailySales
@LoadDate DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF @LoadDate IS NULL SET @LoadDate = CAST(GETUTCDATE() AS DATE);
    BEGIN TRANSACTION;
    BEGIN TRY
        -- Remove existing records for the date (idempotent)
        DELETE FROM dbo.FactSales
        WHERE LoadDate = @LoadDate;

        -- Load from staging
        INSERT INTO dbo.FactSales
        (SalesKey, OrderDate, CustomerKey, ProductKey,
        TerritoryKey, SalesAmount, TaxAmount, Quantity, LoadDate)
        SELECT s.SalesKey, s.OrderDate, s.CustomerKey, s.ProductKey,
        s.TerritoryKey, s.SalesAmount, s.TaxAmount, s.Quantity,
        @LoadDate
        FROM dbo.StagingFactSales s
        WHERE s.LoadDate = @LoadDate;

        COMMIT TRANSACTION;
        PRINT 'Load complete for: ' + CAST(@LoadDate AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        THROW;
    END CATCH;
END;
```

SQL: Check Table Sizes and Column Counts

```
SELECT
s.name AS schema_name,
t.name AS table_name,
COUNT(c.column_id) AS column_count,
t.create_date,
t.modify_date
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.columns c ON t.object_id = c.object_id
WHERE t.is_ms_shipped = 0
GROUP BY s.name, t.name, t.create_date, t.modify_date
ORDER BY column_count DESC;
```

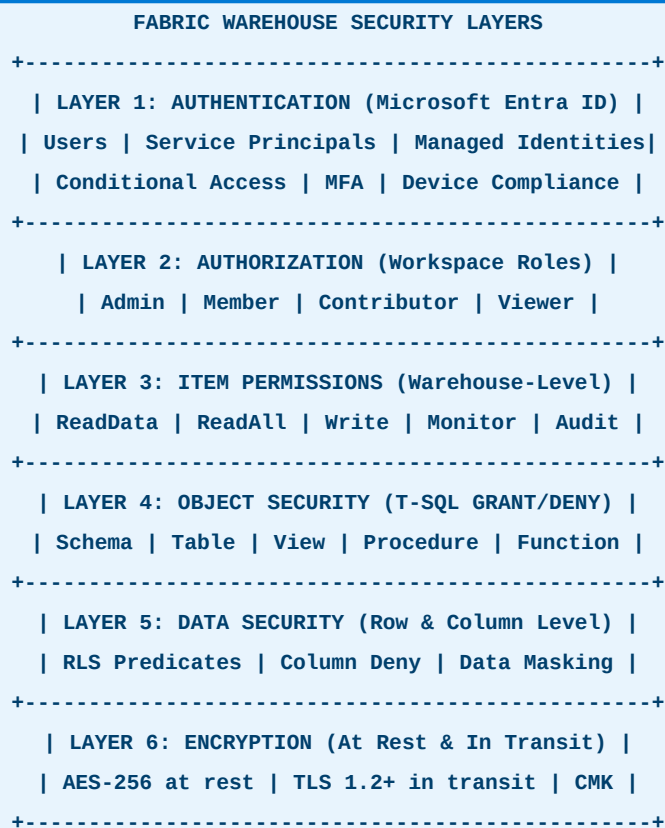
Official Microsoft Documentation References:

- *Create Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/create-warehouse>
- *Tables in Fabric Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/tables>
- *Data Types:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/data-types>
- *T-SQL Surface Area:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/tsql-surface-area>
- *Warehouse Limitations:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/limitations>

05 Warehouse Security

Fabric Warehouse implements a multi-layered security model. Every interaction is encrypted using TLS and authenticated via Microsoft Entra ID (Azure AD). SQL authentication (username/password) is NOT supported — all users must authenticate through Entra ID.

DIAGRAM: Fabric Warehouse Security Architecture (6 Layers)



5.1 Object-Level Security

SQL: Create a Database User from Entra ID

```

-- Create user mapped to Azure AD / Entra ID account
CREATE USER [analyst@contoso.com] FROM EXTERNAL PROVIDER;

-- Create user mapped to a Service Principal
CREATE USER [my-pipeline-spn] FROM EXTERNAL PROVIDER;

-- Create user mapped to an Azure AD Group
CREATE USER [Sales_Analysts_Group] FROM EXTERNAL PROVIDER;

```

SQL: Grant Object-Level Permissions

```
-- Grant SELECT on specific table
GRANT SELECT ON dbo.FactSales TO [analyst@contoso.com];

-- Grant SELECT on entire schema
GRANT SELECT ON SCHEMA::dbo TO [Sales_Analysts_Group];

-- Grant multiple permissions to service principal
GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::dbo TO [my-pipeline-sp];

-- Grant EXECUTE on stored procedure
GRANT EXECUTE ON dbo.usp_LoadDailySales TO [my-pipeline-sp];

-- Deny sensitive table completely (overrides any GRANT)
DENY SELECT ON dbo.SensitivePayroll TO [analyst@contoso.com];
```

SQL: View All Object Permissions

```
SELECT
pr.name AS principal_name,
pr.type_desc AS principal_type,
pe.state_desc AS permission_state,
pe.permission_name,
COALESCE(OBJECT_NAME(pe.major_id),'') AS object_name,
pe.class_desc AS object_class
FROM sys.database_permissions pe
JOIN sys.database_principals pr ON pe.grantee_principal_id = pr.principal_id
WHERE pr.type NOT IN ('R','A') -- exclude roles
ORDER BY pr.name, pe.permission_name;
```

5.2 Row-Level Security (RLS)

RLS restricts which rows a user can see based on their identity. It is transparent to the query — the user writes a normal SELECT and the filter is applied automatically by the security predicate.

SQL: Implement Row-Level Security

```
-- Step 1: Create the security predicate function
CREATE FUNCTION dbo.fn_RLS_RegionFilter (@Region NVARCHAR(50))
RETURNS TABLE WITH SCHEMABINDING
AS RETURN
SELECT 1 AS fn_result
WHERE @Region = USER_NAME() -- match region to login name
OR USER_NAME() = 'sa' -- admin bypass
OR IS_MEMBER('db_owner') = 1; -- owner bypass
GO

-- Step 2: Create security policy using the predicate
CREATE SECURITY POLICY dbo.RegionSalesPolicy
ADD FILTER PREDICATE dbo.fn_RLS_RegionFilter(Region)
ON dbo.FactSales
WITH (STATE = ON);
GO

-- Test: User 'North' sees only North region rows
EXECUTE AS USER = 'North';
SELECT COUNT(*) AS rows_visible FROM dbo.FactSales;
REVERT;
```

5.3 Column-Level Security

SQL: Deny Access to Sensitive Columns

```
-- Deny specific sensitive columns to analyst role
DENY SELECT ON dbo.DimEmployee (Salary, BankAccount, SSN, DateOfBirth)
TO [analyst@contoso.com];

-- Verify: analyst can query table but NOT the denied columns
-- SELECT Salary FROM dbo.DimEmployee; -- returns error
-- SELECT EmployeeName FROM dbo.DimEmployee; -- succeeds
```

5.4 Dynamic Data Masking

SQL: Add Dynamic Data Masking to Sensitive Columns

```
-- Add masking to email column (show first char, then ***@domain)
ALTER TABLE dbo.DimCustomer
ALTER COLUMN Email
ADD MASKED WITH (FUNCTION = 'email()');

-- Partial masking for phone number
ALTER TABLE dbo.DimCustomer
ALTER COLUMN PhoneNumber
ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)');

-- Default masking (shows XXXX for strings, 0 for numbers)
ALTER TABLE dbo.DimEmployee
ALTER COLUMN Salary
ADD MASKED WITH (FUNCTION = 'default()');

-- Grant UNMASK permission to privileged users
GRANT UNMASK ON dbo.DimCustomer TO [hr_manager@contoso.com];
```

SQL: Audit: Review User Permissions Summary

```
-- Complete permissions audit for a specific user
SELECT
'Object Permission' AS permission_type,
pe.permission_name,
pe.state_desc,
OBJECT_NAME(pe.major_id) AS object_name,
pe.class_desc
FROM sys.database_permissions pe
JOIN sys.database_principals pr
ON pe.grantee_principal_id = pr.principal_id
WHERE pr.name = 'analyst@contoso.com'
UNION ALL
SELECT
'Role Membership',
r.name, 'IS MEMBER', NULL, 'DATABASE_ROLE'
FROM sys.database_role_members rm
JOIN sys.database_principals r
ON rm.role_principal_id = r.principal_id
JOIN sys.database_principals m
ON rm.member_principal_id = m.principal_id
WHERE m.name = 'analyst@contoso.com';
```

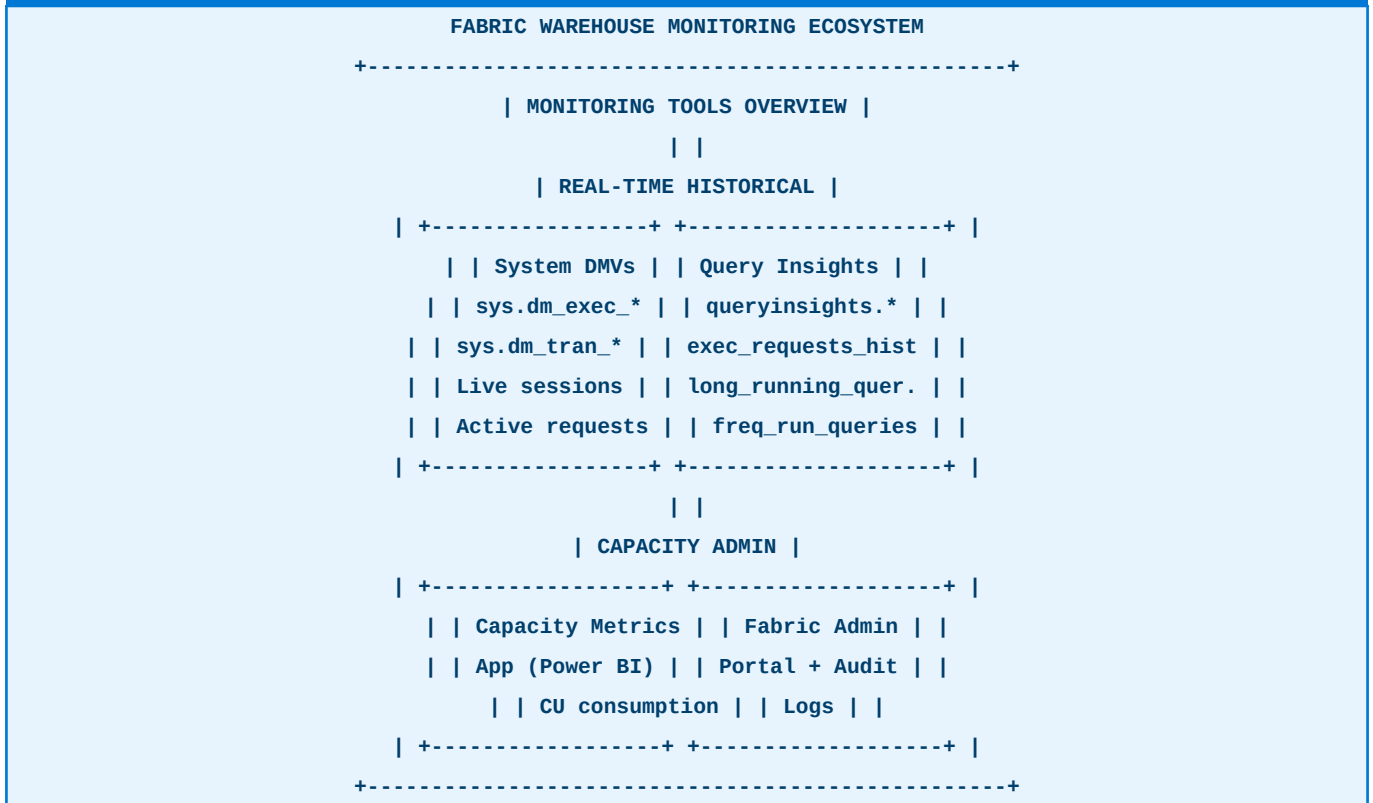

Official Microsoft Documentation References:

- *Secure Fabric Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/security>
- *SQL Granular Permissions:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/sql-granular-permissions>
- *Row-Level Security:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/row-level-security>
- *Dynamic Data Masking:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/dynamic-data-masking>
- *Column-Level Security:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/column-level-security>
- *Entra ID Authentication:* <https://learn.microsoft.com/en-us/fabric/security/security-fundamentals>
- *Share Warehouse Permissions:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/share-warehouse-manage-permissions>

06 Fabric Warehouse Capacity & Monitoring

Monitoring is a core DBA responsibility in Fabric Warehouse. Microsoft provides two complementary monitoring surfaces: Dynamic Management Views (DMVs) for real-time operational data, and the Query Insights schema for historical analytics. Additionally, the Fabric Capacity Metrics App tracks CU consumption.

DIAGRAM: Fabric Warehouse Monitoring Architecture



6.1 System DMVs — Real-Time Monitoring

DMV	What It Shows	Required Permission
sys.dm_exec_connections	Active connections, client IP, protocol	Monitor or Admin
sys.dm_exec_sessions	All sessions, resource usage, login info	Monitor or Admin
sys.dm_exec_requests	Currently running queries, wait types	Monitor or Admin
sys.dm_tran_active_transactions	Open transactions and age	Admin
sys.dm_tran_session_transactions	Session-to-transaction mapping	Admin

SQL: Monitor Active Connections

```
SELECT
c.session_id,
c.client_net_address,
c.client_tcp_port,
c.auth_scheme,
c.protocol_type,
c.connect_time,
s.login_name,
s.status AS session_status
FROM sys.dm_exec_connections c
JOIN sys.dm_exec_sessions s ON c.session_id = s.session_id
WHERE s.is_user_process = 1
ORDER BY c.connect_time DESC;
```

SQL: Find Long-Running Queries (Over 30 Seconds)

```
SELECT
r.session_id,
s.login_name,
r.status,
r.total_elapsed_time / 1000.0 AS elapsed_sec,
r.wait_type,
r.wait_time / 1000.0 AS wait_sec,
r.blocking_session_id,
r.cpu_time,
r.logical_reads,
r.command
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
WHERE r.total_elapsed_time > 30000 -- > 30 seconds
AND r.session_id > 50
ORDER BY r.total_elapsed_time DESC;
```

SQL: Detect and Kill Blocking Sessions

```
-- Find blocking chains
SELECT
blocker.session_id AS blocker_id,
bs.login_name AS blocker_login,
blocked.session_id AS blocked_id,
ds.login_name AS blocked_login,
blocked.wait_type,
blocked.wait_time/1000.0 AS blocked_wait_sec
FROM sys.dm_exec_requests blocked
JOIN sys.dm_exec_requests blocker
ON blocked.blocking_session_id = blocker.session_id
JOIN sys.dm_exec_sessions bs ON blocker.session_id = bs.session_id
JOIN sys.dm_exec_sessions ds ON blocked.session_id = ds.session_id;

-- Kill a session (Workspace Admin only)
-- KILL 123; -- Replace 123 with actual session_id
```

6.2 Query Insights Schema

View	Content	Best Use
queryinsights.exec_requests_history	All completed/failed/cancelled queries	Audit, troubleshoot, SLA reporting
queryinsights.exec_sessions_history	Historical session resource data	User resource attribution
queryinsights.long_running_queries	Aggregated slow query statistics	Weekly performance review
queryinsights.frequently_run_queries	Most repeated query patterns	Caching and optimization targets

SQL: Query History — Last 24 Hours

```
SELECT TOP 100
  login_name,
  start_time,
  end_time,
  status,
  total_elapsed_time_ms / 1000.0 AS elapsed_sec,
  row_count,
  command
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(HOUR, -24, GETUTCDATE())
ORDER BY start_time DESC;
```

SQL: Failed Queries Investigation

```
SELECT
  login_name,
  start_time,
  status,
  total_elapsed_time_ms/1000.0 AS elapsed_sec,
  command
FROM queryinsights.exec_requests_history
WHERE status IN ('Failed','Cancelled')
AND start_time >= DATEADD(DAY, -7, GETUTCDATE())
ORDER BY start_time DESC;
```

SQL: Top 20 Slowest Queries This Week

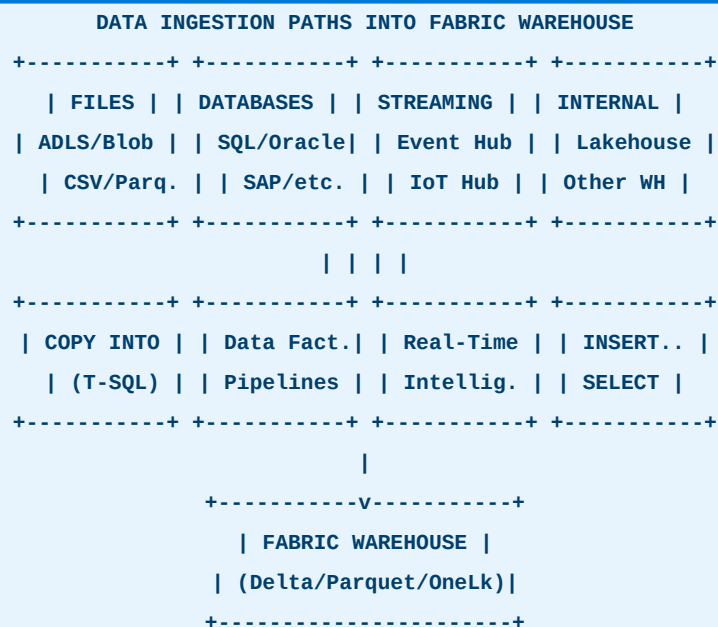
```
SELECT TOP 20
  command,
  login_name,
  median_total_elapsed_time_ms/1000.0 AS median_sec,
  avg_total_elapsed_time_ms/1000.0 AS avg_sec,
  max_total_elapsed_time_ms/1000.0 AS max_sec,
  number_of_iterations AS run_count
FROM queryinsights.long_running_queries
ORDER BY median_total_elapsed_time_ms DESC;
```

- Official Microsoft Documentation References:
- *Monitoring Overview:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/monitoring-overview>
 - *DMV Monitoring:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/monitor-using-dmv>
 - *Query Insights:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/query-insights>
 - *Capacity Metrics App:* <https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app>

07 Data Loading & Ingestion

Loading data into Fabric Warehouse efficiently is one of the most critical DBA skills. Fabric supports multiple ingestion paths — the choice depends on source type, volume, latency requirements, and transformation complexity.

DIAGRAM: Data Ingestion Architecture



7.1 COPY INTO — Bulk Load Command

SQL: COPY INTO from Parquet Files (ADLS Gen2)

```
-- Most efficient bulk load method
COPY INTO dbo.FactSales
(OrderDate, CustomerKey, ProductKey, TerritoryKey, SalesAmount, Quantity)
FROM 'https://dfs.core.windows.net/sales/2024/*.parquet'
WITH (
  FILE_TYPE = 'PARQUET',
  CREDENTIAL = (IDENTITY = 'Managed Identity')
);
```

SQL: COPY INTO from CSV with Options

```
COPY INTO dbo.StagingOrders
FROM 'https://blob.core.windows.net/raw/orders/*.csv'
WITH (
  FILE_TYPE = 'CSV',
  FIRSTROW = 2, -- skip header row
  FIELDTERMINATOR = ',',
  ROWTERMINATOR = '0x0A',
  DATEFORMAT = 'ymd',
  ENCODING = 'UTF8',
  CREDENTIAL = (IDENTITY = 'Managed Identity')
);
```

SQL: Staging + Atomic Load Pattern

```
-- Phase 1: Load raw data into staging (fast, no constraints)
COPY INTO dbo.StagingFactSales
FROM 'https://.dfs.core.windows.net/raw/daily/*.parquet'
WITH (FILE_TYPE='PARQUET', CREDENTIAL=(IDENTITY='Managed Identity'));

-- Phase 2: Validate staging
SELECT COUNT(*) AS staged_rows,
SUM(CASE WHEN CustomerKey IS NULL THEN 1 ELSE 0 END) AS null_customers
FROM dbo.StagingFactSales;

-- Phase 3: Atomic load into target
BEGIN TRANSACTION;
BEGIN TRY
INSERT INTO dbo.FactSales
SELECT OrderDate, CustomerKey, ProductKey, TerritoryKey,
SalesAmount, TaxAmount, Quantity, CAST(GETUTCDATE() AS DATE)
FROM dbo.StagingFactSales
WHERE CustomerKey IS NOT NULL
AND SalesAmount >= 0;

TRUNCATE TABLE dbo.StagingFactSales; -- clear staging
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
ROLLBACK; THROW;
END CATCH;
```

SQL: Incremental Load with Watermark

```
-- Identify new records since last load
DECLARE @LastLoadDate DATE;
SELECT @LastLoadDate = MAX(LoadDate) FROM dbo.FactSales;

-- Load only records newer than watermark
INSERT INTO dbo.FactSales
(OrderDate, CustomerKey, ProductKey, SalesAmount, LoadDate)
SELECT s.OrderDate, s.CustomerKey, s.ProductKey, s.SalesAmount,
CAST(GETUTCDATE() AS DATE)
FROM dbo.StagingFactSales s
WHERE s.OrderDate > ISNULL(@LastLoadDate, '1900-01-01');
```

SQL: UPSERT Pattern with MERGE

```
MERGE dbo.DimCustomer AS tgt
USING dbo.StagingCustomer AS src
ON tgt.CustomerKey = src.CustomerKey
WHEN MATCHED AND (
    tgt.CustomerName <> src.CustomerName OR
    tgt.Region <> src.Region OR
    tgt.Email <> src.Email
) THEN
UPDATE SET
    tgt.CustomerName = src.CustomerName,
    tgt.Region = src.Region,
    tgt.Email = src.Email,
    tgt.ModifiedDate = GETUTCDATE()
WHEN NOT MATCHED BY TARGET THEN
INSERT (CustomerKey, CustomerName, Region, Email, CreatedDate)
VALUES (src.CustomerKey, src.CustomerName, src.Region,
    src.Email, GETUTCDATE());
```

Official Microsoft Documentation References:

- *COPY INTO T-SQL*: <https://learn.microsoft.com/en-us/sql/t-sql/statements/copy-into-transact-sql?view=fabric>
- *Ingest Data with Pipelines*: <https://learn.microsoft.com/en-us/fabric/data-warehouse/ingest-data-pipelines>
- *Ingest Data Overview*: <https://learn.microsoft.com/en-us/fabric/data-warehouse/ingest-data>
- *Best Practices for Loading*: <https://learn.microsoft.com/en-us/fabric/data-warehouse/guidelines-warehouse-performance>

08 Transactions, Locking & Concurrency

Fabric Warehouse supports full ACID transactions built on Delta Lake's atomic commit protocol. Snapshot Isolation is the **ONLY** supported isolation level. This means readers never block writers, and writers never block readers — but two concurrent writers on the same table can conflict.

DIAGRAM: Snapshot Isolation & Write-Write Conflict

```

      SNAPSHOT ISOLATION MODEL

      Time --->

      T=0 T=1 T=2 T=3 T=4
      | | | | |
      | Session1: BEGIN TRAN |
      | | UPDATE FactSales | | |
      | | | | |
      | | Session2: BEGIN TRAN |
      | | | UPDATE FactSales |
      | | | COMMIT S2 |
      | | | | |
      | | | | COMMIT S1 -- FAILS!
      | (write-write conflict)
      RESULT: Session 2 wins (committed first)
              Session 1 must retry

      KEY RULE: LAST WRITER to COMMIT wins.
      INSERTS rarely conflict (new Parquet files).
      UPDATES/DELETES CAN conflict (same Delta log).
  
```

SQL: Explicit Transaction with Error Handling

```

BEGIN TRANSACTION;
BEGIN TRY
  -- Multi-table atomic operation
  INSERT INTO dbo.FactSales
  (SalesKey, OrderDate, CustomerKey, SalesAmount)
  VALUES (NEXT VALUE FOR dbo.seq_SalesKey,
  '2024-06-15', 1001, 2500.00);

  UPDATE dbo.CustomerSummary
  SET TotalPurchases = TotalPurchases + 2500.00,
  LastOrderDate = '2024-06-15'
  WHERE CustomerKey = 1001;

  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
  DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
  DECLARE @ErrSev INT = ERROR_SEVERITY();
  RAISERROR(@ErrMsg, @ErrSev, 1);
END CATCH;
  
```


SQL: Retry Pattern for Write-Write Conflicts

```
DECLARE @retry INT = 0;
DECLARE @maxRetry INT = 3;
DECLARE @success BIT = 0;

WHILE @retry < @maxRetry AND @success = 0
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE dbo.FactSales
        SET SalesAmount = SalesAmount * 1.10
        WHERE OrderDate = '2024-06-15';

        COMMIT TRANSACTION;

        SET @success = 1; -- exit loop on success
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        SET @retry += 1;
        IF @retry = @maxRetry THROW; -- rethrow after max retries
        WAITFOR DELAY '00:00:02'; -- wait 2 sec before retry
    END CATCH;
END;
```

SQL: Monitor Open Transactions

```
-- Find sessions with open transactions older than 60 seconds
SELECT
    st.session_id,
    s.login_name,
    t.transaction_begin_time,
    DATEDIFF(SECOND,t.transaction_begin_time,GETUTCDATE()) AS age_sec,
    t.transaction_type
FROM sys.dm_tran_session_transactions st
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
JOIN sys.dm_tran_active_transactions t ON st.transaction_id = t.transaction_id
WHERE DATEDIFF(SECOND,t.transaction_begin_time,GETUTCDATE()) > 60
ORDER BY age_sec DESC;
```

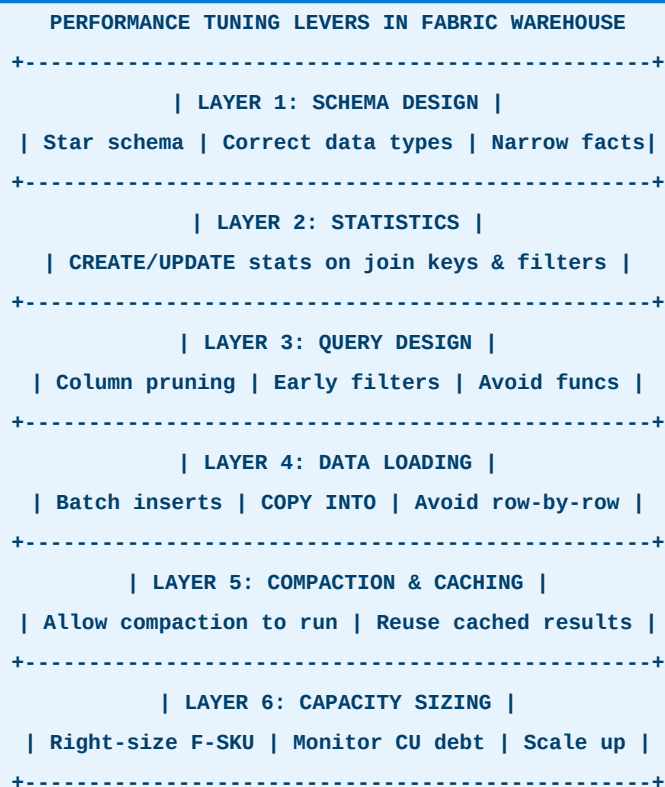
Official Microsoft Documentation References:

- *Transactions in Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/transactions>
- *Workload Management:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/workload-management>
- *Concurrency in Fabric:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/statistics>

09 Performance Tuning & Optimization

Performance tuning in Fabric Warehouse differs from SQL Server tuning because the engine uses a columnar Parquet storage format with distributed MPP processing. There are no traditional indexes — instead, statistics, schema design, query patterns, and data compaction are the primary tuning levers.

DIAGRAM: Performance Tuning Pyramid



9.1 Statistics Management

SQL: Create Statistics on Key Columns

```

-- Create stats on JOIN key (critical for query optimizer)
CREATE STATISTICS stat_FactSales_CustomerKey
ON dbo.FactSales (CustomerKey);

-- Create stats on filter column (date range queries)
CREATE STATISTICS stat_FactSales_OrderDate
ON dbo.FactSales (OrderDate);

-- Multi-column stats for correlated filter patterns
CREATE STATISTICS stat_FactSales_Region_Date
ON dbo.FactSales (Region, OrderDate);

-- Update all stats after major data loads
UPDATE STATISTICS dbo.FactSales;
UPDATE STATISTICS dbo.DimCustomer;
  
```

SQL: Statistics Health Audit

```
-- Find stale statistics (not updated in 7+ days)
SELECT
t.name AS table_name,
s.name AS stat_name,
STATS_DATE(s.object_id, s.stats_id) AS last_updated,
DATEDIFF(DAY,
STATS_DATE(s.object_id, s.stats_id),
GETUTCDATE()) AS days_old,
s.auto_created
FROM sys.stats s
JOIN sys.tables t ON s.object_id = t.object_id
WHERE t.is_ms_shipped = 0
AND (STATS_DATE(s.object_id,s.stats_id) IS NULL
OR STATS_DATE(s.object_id,s.stats_id) < DATEADD(DAY,-7,GETUTCDATE()))
ORDER BY days_old DESC NULLS FIRST;
```

SQL: Query Performance Anti-Patterns to Avoid

```
-- BAD: Function on filter column (disables predicate pushdown)
-- WHERE YEAR(OrderDate) = 2024 <-- AVOID

-- GOOD: Range filter allows Parquet row-group elimination
SELECT CustomerKey, SalesAmount
FROM dbo.FactSales
WHERE OrderDate >= '2024-01-01'
AND OrderDate < '2025-01-01';

-- BAD: SELECT * reads all Parquet columns
-- SELECT * FROM dbo.FactSales <-- AVOID

-- GOOD: Column pruning -- only reads needed columns
SELECT OrderDate, CustomerKey, SalesAmount
FROM dbo.FactSales
WHERE Region = 'North';
```

SQL: Weekly Performance Review Queries

```
-- 1. Top 10 slowest queries this week
SELECT TOP 10 command, login_name,
total_elapsed_time_ms/1000.0 AS elapsed_sec,
start_time
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY,-7,GETUTCDATE())
ORDER BY total_elapsed_time_ms DESC;

-- 2. Query volume by day
SELECT CAST(start_time AS DATE) AS query_date,
COUNT(*) AS query_count,
AVG(total_elapsed_time_ms)/1000.0 AS avg_sec
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY,-7,GETUTCDATE())
GROUP BY CAST(start_time AS DATE)
ORDER BY query_date;
```

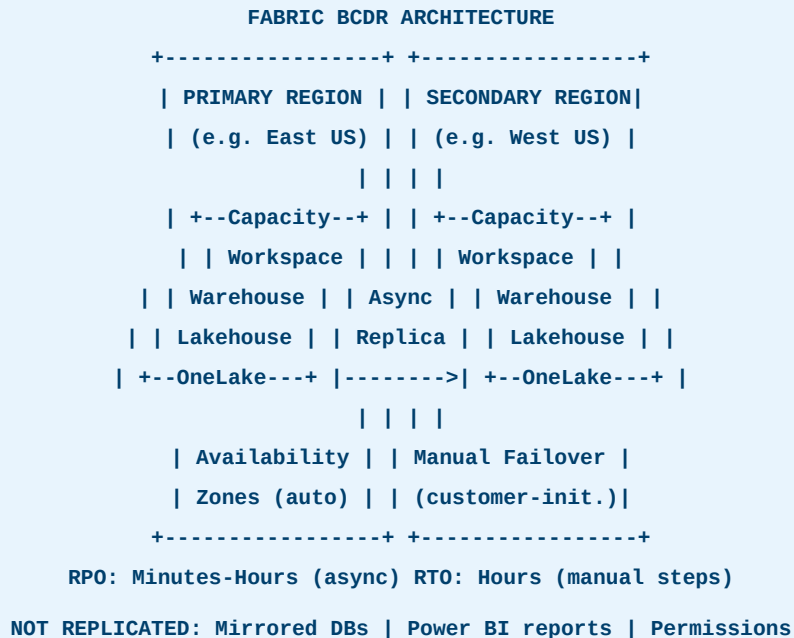
Official Microsoft Documentation References:

- *Performance Guidelines:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/guidelines-warehouse-performance>
- *Statistics in Fabric:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/statistics>
- *Query Insights:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/query-insights>
- *Autotune Query Optimization:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/autotune>

10 Disaster Recovery & Business Continuity

Fabric Warehouse uses a shared responsibility model for business continuity. Microsoft manages infrastructure availability, zone-level redundancy, and the platform restore tooling. Customers are responsible for enabling BCDR, configuring cross-region strategies, and executing recovery procedures.

DIAGRAM: Fabric BCDR & Regional Replication



10.1 BCDR Monitoring & Validation Queries

SQL: Check Data Freshness After Failover

```
-- Validate data freshness in recovered Warehouse
SELECT
'FactSales' AS table_name,
COUNT(*) AS total_rows,
MAX(OrderDate) AS latest_order_date,
MAX(LoadDate) AS latest_load_date,
GETUTCDATE() AS current_utc,
DATEDIFF(HOUR,
MAX(LoadDate),
GETUTCDATE()) AS hours_since_last_load
FROM dbo.FactSales
UNION ALL
SELECT
'DimCustomer',
COUNT(*),
NULL,
MAX(ModifiedDate),
GETUTCDATE(),
DATEDIFF(HOUR, MAX(ModifiedDate), GETUTCDATE())
FROM dbo.DimCustomer;
```

SQL: Validate Object Counts After Recovery

```
-- Quick inventory check post-recovery
SELECT
  'Tables' AS object_type,
  COUNT(*) AS object_count
FROM sys.tables WHERE is_ms_shipped = 0
UNION ALL
SELECT 'Views', COUNT(*) FROM sys.views WHERE is_ms_shipped = 0
UNION ALL
SELECT 'Procs', COUNT(*) FROM sys.procedures WHERE is_ms_shipped = 0
UNION ALL
SELECT 'Functions',COUNT(*) FROM sys.objects
WHERE type IN ('FN','IF','TF') AND is_ms_shipped = 0;
```

SQL: Recreate Permissions Post-Failover Script Template

```
-- Run this script on recovered Warehouse to restore permissions
-- Step 1: Recreate users
CREATE USER [analyst@contoso.com] FROM EXTERNAL PROVIDER;
CREATE USER [my-pipeline-spn] FROM EXTERNAL PROVIDER;
CREATE USER [Sales_Team_Group] FROM EXTERNAL PROVIDER;

-- Step 2: Restore grants
GRANT SELECT ON SCHEMA::dbo TO [Sales_Team_Group];
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo
TO [my-pipeline-spn];

-- Step 3: Restore RLS policies (re-run original RLS creation scripts)
-- Step 4: Restore DDM masking (re-run ALTER TABLE masking scripts)
-- Step 5: Validate with test queries
EXECUTE AS USER = 'analyst@contoso.com';
SELECT TOP 5 * FROM dbo.DimCustomer;
REVERT;
```

Official Microsoft Documentation References:

- *BCDR Overview*: <https://learn.microsoft.com/en-us/fabric/security/experience-specific-guidance>
- *Reliability in Fabric*: <https://learn.microsoft.com/en-us/azure/reliability/reliability-fabric>
- *Restore Warehouse*: <https://learn.microsoft.com/en-us/fabric/data-warehouse/restore-in-place>
- *Disaster Recovery Guidance*: <https://learn.microsoft.com/en-us/fabric/security/experience-specific-guidance>

11 Source Control, CI/CD & DevOps

Source control and CI/CD pipelines are essential for professional Fabric Warehouse administration. Git integration allows entire workspace configurations to be version-controlled, enabling automated deployments and rollback capabilities.

DIAGRAM: CI/CD Pipeline for Fabric Warehouse



11.1 DBA SQL — Idempotent Schema Scripts (CI/CD Safe)

SQL: Idempotent View Creation (CREATE OR ALTER)

```

-- Safe to run multiple times in any environment
CREATE OR ALTER VIEW dbo.vw_MonthlySalesSummary AS
SELECT
YEAR(f.OrderDate) AS SalesYear,
MONTH(f.OrderDate) AS SalesMonth,
c.Region,
c.Country,
SUM(f.SalesAmount) AS TotalSales,
COUNT(DISTINCT f.CustomerKey) AS UniqueCustomers,
SUM(f.Quantity) AS TotalUnits,
AVG(f.SalesAmount) AS AvgOrderValue
FROM dbo.FactSales f
JOIN dbo.DimCustomer c ON f.CustomerKey = c.CustomerKey
GROUP BY YEAR(f.OrderDate), MONTH(f.OrderDate), c.Region, c.Country;
  
```

SQL: Idempotent Procedure Creation

```
CREATE OR ALTER PROCEDURE dbo.usp_RefreshDimensionStats
@TableName NVARCHAR(128)
AS
BEGIN
SET NOCOUNT ON;
DECLARE @sql NVARCHAR(500) =
'UPDATE STATISTICS dbo.' + QUOTENAME(@TableName);
EXEC sp_executesql @sql;
PRINT 'Statistics updated for: ' + @TableName;
END;
```

SQL: Schema Deployment Validation Query

```
-- Run after each deployment to validate schema integrity
SELECT
'TABLES' AS type,
s.name AS schema_name,
t.name AS object_name,
t.modify_date
FROM sys.tables t JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE t.is_ms_shipped=0
UNION ALL
SELECT 'VIEWS', s.name, v.name, v.modify_date
FROM sys.views v JOIN sys.schemas s ON v.schema_id=s.schema_id
WHERE v.is_ms_shipped=0
UNION ALL
SELECT 'PROCS', s.name, p.name, p.modify_date
FROM sys.procedures p JOIN sys.schemas s ON p.schema_id=s.schema_id
WHERE p.is_ms_shipped=0
ORDER BY modify_date DESC;
```

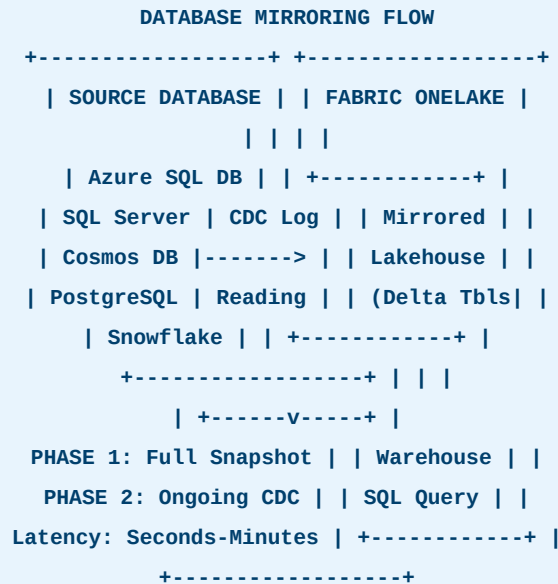
Official Microsoft Documentation References:

- *Source Control for Warehouse:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/source-control>
- *Git Integration:* <https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration>
- *Deployment Pipelines:* <https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines>
- *Fabric REST API:* <https://learn.microsoft.com/en-us/rest/api/fabric/articles/using-fabric-apis>

12 Database Mirroring & Real-Time Sync

Fabric Database Mirroring enables near-real-time continuous replication from external operational databases into Fabric OneLake using Change Data Capture (CDC). Data lands as Delta Lake tables, immediately queryable by Warehouses and Lakehouses without ETL pipeline code.

DIAGRAM: Database Mirroring Architecture



12.1 Enable CDC on Azure SQL Database (Source Side)

SQL: Enable CDC on Azure SQL Database

```

-- Run on SOURCE Azure SQL Database (not Fabric)
-- Step 1: Enable CDC at database level
EXEC sys.sp_cdc_enable_db;

-- Step 2: Enable CDC on each table
EXEC sys.sp_cdc_enable_table
@source_schema = 'dbo',
@source_name = 'Orders',
@role_name = NULL,
@supports_net_changes = 1;

EXEC sys.sp_cdc_enable_table
@source_schema = 'dbo',
@source_name = 'Customers',
@role_name = NULL;

-- Step 3: Verify CDC is enabled
SELECT name, is_cdc_enabled
FROM sys.databases
WHERE name = DB_NAME();

-- Step 4: Verify enabled tables
SELECT source_schema, source_name, is_tracked_by_cdc
FROM cdc.change_tables;
  
```

SQL: Monitor Mirroring Data Freshness (Fabric Side)

```
-- Run in Fabric Warehouse to check mirrored data freshness
SELECT
'MirroredOrders' AS mirrored_table,
COUNT(*) AS total_rows,
MAX(LastModifiedDate) AS latest_record,
GETUTCDATE() AS current_utc,
DATEDIFF(MINUTE,
MAX(LastModifiedDate),
GETUTCDATE()) AS lag_minutes
FROM SourceDB_Mirror.dbo.Orders;
-- If lag > 10 minutes, investigate CDC agent on source
```

SQL: Cross-Database Query: Mirror + Warehouse

```
-- Join mirrored operational data with Warehouse dimension
SELECT
o.OrderID,
o.OrderDate,
o.TotalAmount,
c.CustomerName,
c.Region,
p.ProductName,
p.Category
FROM SourceDB_Mirror.dbo.Orders o -- mirrored table
JOIN SourceDB_Mirror.dbo.OrderDetails od -- mirrored table
ON o.OrderID = od.OrderID
JOIN dbo.DimCustomer c -- Warehouse dimension
ON o.CustomerID = c.CustomerID
JOIN dbo.DimProduct p -- Warehouse dimension
ON od.ProductID = p.ProductID
WHERE o.OrderDate >= DATEADD(HOUR, -1, GETUTCDATE());
```

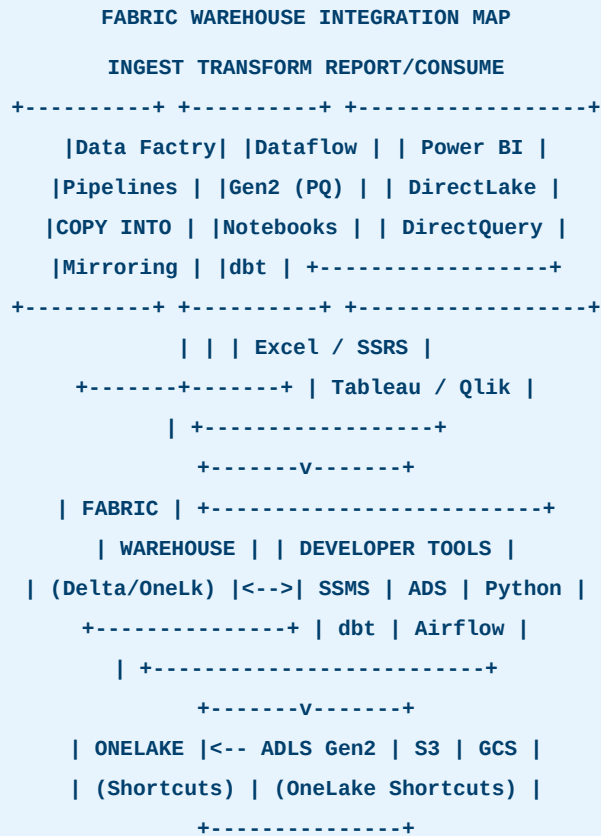
Official Microsoft Documentation References:

- *What is Mirroring:* <https://learn.microsoft.com/en-us/fabric/database/mirrored-database/overview>
- *Mirror Azure SQL Database:* <https://learn.microsoft.com/en-us/fabric/database/mirrored-database/azure-sql-database>
- *Mirror SQL Server:* <https://learn.microsoft.com/en-us/fabric/database/mirrored-database/sql-server>
- *Mirroring Limitations:* <https://learn.microsoft.com/en-us/fabric/database/mirrored-database/limitations>

13 Integration with External Services & Tools

Fabric Warehouse integrates natively with all Microsoft data and analytics services and connects to a wide range of third-party tools via standard SQL endpoints. Understanding these integrations allows DBAs to design optimal end-to-end data architectures.

DIAGRAM: Fabric Warehouse Integration Ecosystem



13.1 Power BI Integration SQL Patterns

SQL: Create Optimized Reporting Views for Power BI

```
-- Pre-aggregated view reduces Power BI query complexity
CREATE OR ALTER VIEW dbo.vw_BI_SalesByRegionMonth AS
SELECT
YEAR(f.OrderDate) AS Year,
MONTH(f.OrderDate) AS Month,
DATENAME(MONTH,f.OrderDate) AS MonthName,
c.Region,
c.Country,
c.Segment,
p.Category,
COUNT(DISTINCT f.SalesKey) AS OrderCount,
COUNT(DISTINCT f.CustomerKey) AS UniqueCustomers,
SUM(f.SalesAmount) AS TotalSales,
SUM(f.TaxAmount) AS TotalTax,
AVG(f.SalesAmount) AS AvgOrderValue
FROM dbo.FactSales f
JOIN dbo.DimCustomer c ON f.CustomerKey = c.CustomerKey
JOIN dbo.DimProduct p ON f.ProductKey = p.ProductKey
GROUP BY
YEAR(f.OrderDate), MONTH(f.OrderDate),
DATENAME(MONTH,f.OrderDate),
c.Region, c.Country, c.Segment, p.Category;
```

13.2 Cross-Database Queries (Warehouse + Lakehouse)**SQL: Query Lakehouse Tables from Warehouse**

```
-- No additional setup needed -- same workspace automatic access
-- Syntax: [LakehouseName].[schema].[table]
SELECT
w.OrderDate,
w.SalesAmount,
l.CustomerSegment, -- column from Lakehouse
l.LifetimeValue, -- column from Lakehouse
l.ChurnRiskScore -- ML prediction from Lakehouse
FROM dbo.FactSales w -- Warehouse
JOIN AnalyticsLakehouse.dbo.Customers l -- Lakehouse
ON w.CustomerKey = l.CustomerKey
WHERE w.OrderDate >= DATEADD(DAY, -30, GETUTCDATE());
```

13.3 SSMS & Azure Data Studio Connection

CONNECTION: Connection String: Server = .datawarehouse.fabric.microsoft.com | Database = | Authentication = Active Directory Interactive | Port = 1433 | Encrypt = Mandatory

SQL: Validate SSMS Connection

```
-- Run this in SSMS after connecting to verify Fabric Warehouse
SELECT
@@SERVERNAME AS server_endpoint,
DB_NAME() AS current_database,
USER_NAME() AS current_user,
GETUTCDATE() AS server_utc_time,
@@VERSION AS engine_version;
```

SQL: Python Connection Example (for DBAs running scripts)

```
-- Documentation reference for Python connectivity:
-- pip install pyodbc
--
-- import pyodbc
-- conn_str = (
-- 'DRIVER={ODBC Driver 18 for SQL Server};'
-- 'SERVER=.datawarehouse.fabric.microsoft.com;'
-- 'DATABASE=;'
-- 'Authentication=ActiveDirectoryInteractive;'
-- )
-- conn = pyodbc.connect(conn_str)
-- cursor = conn.cursor()
-- cursor.execute('SELECT TOP 10 * FROM dbo.FactSales')
-- rows = cursor.fetchall()

-- Equivalent T-SQL to validate data from SSMS/ADS:
SELECT TOP 10 * FROM dbo.FactSales
ORDER BY OrderDate DESC;
```

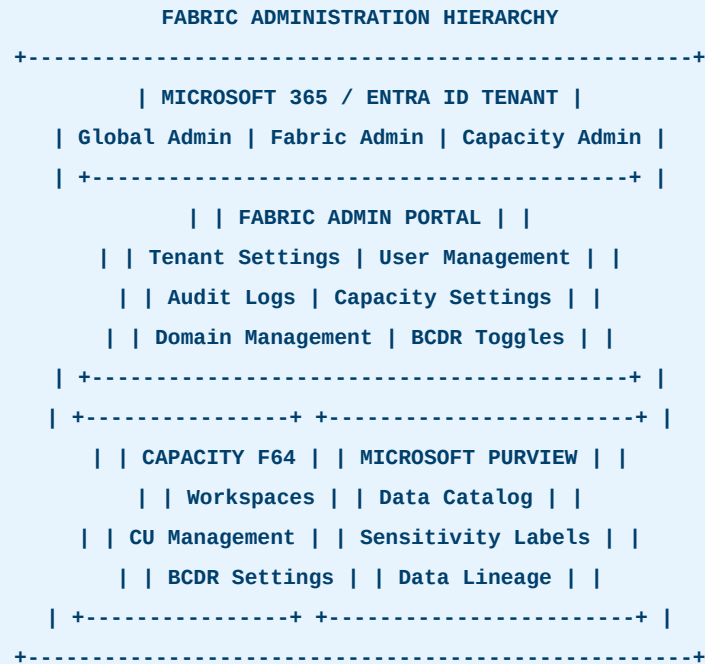
Official Microsoft Documentation References:

- *Connect with SSMS:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/connectivity>
- *Power BI Integration:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/create-reports>
- *Cross-Database Queries:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/query-warehouse>
- *OneLake Shortcuts:* <https://learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts>
- *dbt with Fabric:* <https://learn.microsoft.com/en-us/fabric/data-warehouse/tutorial-setup-dbt>

14 Fabric Administration & Governance

Fabric administrators manage the platform at the tenant and capacity levels through the Fabric Admin Portal. Administration includes tenant settings, capacity management, user activity monitoring, governance via Microsoft Purview, and compliance with data residency and regulatory requirements.

DIAGRAM: Fabric Administration Architecture



14.1 DBA Administration SQL Queries

SQL: Full Database Object Inventory (DBA Audit)

```

-- Complete object inventory for governance documentation
SELECT
o.type_desc AS object_type,
s.name AS schema_name,
o.name AS object_name,
o.create_date,
o.modify_date,
OBJECTPROPERTY(o.object_id, 'IsMSShipped') AS is_system
FROM sys.objects o
JOIN sys.schemas s ON o.schema_id = s.schema_id
WHERE o.type IN ('U', 'V', 'P', 'FN', 'IF', 'TF') -- tables, views, procs, funcs
AND OBJECTPROPERTY(o.object_id, 'IsMSShipped') = 0
ORDER BY o.type_desc, s.name, o.name;
  
```

SQL: Column Lineage Metadata Report

```
-- Document all columns with data types for governance
SELECT
s.name AS schema_name,
t.name AS table_name,
c.column_id,
c.name AS column_name,
tp.name AS data_type,
c.max_length,
c.precision,
c.scale,
c.is_nullable,
c.is_identity,
c.is_computed,
dc.definition AS default_value
FROM sys.columns c
JOIN sys.tables t ON c.object_id = t.object_id
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.types tp ON c.user_type_id = tp.user_type_id
LEFT JOIN sys.default_constraints dc
ON c.default_object_id = dc.object_id
WHERE t.is_ms_shipped = 0
ORDER BY s.name, t.name, c.column_id;
```

SQL: Security Audit: All Users and Their Permissions

```
-- Comprehensive security audit report
SELECT
pr.name AS principal_name,
pr.type_desc AS principal_type,
pe.class_desc AS permission_class,
pe.permission_name,
pe.state_desc AS permission_state,
CASE pe.class
WHEN 1 THEN OBJECT_NAME(pe.major_id)
WHEN 3 THEN SCHEMA_NAME(pe.major_id)
ELSE NULL
END AS object_name
FROM sys.database_permissions pe
JOIN sys.database_principals pr
ON pe.grantee_principal_id = pr.principal_id
WHERE pr.type NOT IN ('R') -- exclude built-in roles
ORDER BY pr.name, pe.permission_name;
```

SQL: Activity Audit — Recent DDL Changes

```
-- Find recently modified objects (schema changes in last 30 days)
SELECT
o.type_desc AS object_type,
s.name AS schema_name,
o.name AS object_name,
o.create_date,
o.modify_date,
DATEDIFF(DAY,o.modify_date,GETUTCDATE()) AS days_since_change
FROM sys.objects o
JOIN sys.schemas s ON o.schema_id = s.schema_id
WHERE o.is_ms_shipped = 0
AND o.modify_date >= DATEADD(DAY, -30, GETUTCDATE())
ORDER BY o.modify_date DESC;
```

Official Microsoft Documentation References:

- *Fabric Admin Portal:* <https://learn.microsoft.com/en-us/fabric/admin/admin-overview>
- *Tenant Settings:* <https://learn.microsoft.com/en-us/fabric/admin/tenant-settings-index>
- *Audit Logs:* <https://learn.microsoft.com/en-us/fabric/admin/track-user-activities>
- *Microsoft Purview:* <https://learn.microsoft.com/en-us/fabric/governance/microsoft-purview-fabric>
- *Domain Management:* <https://learn.microsoft.com/en-us/fabric/governance/domains>
- *Fabric Certification DP-600:* <https://learn.microsoft.com/en-us/credentials/certifications/fabric-analytics-engineer-associate/>

15 DBA Master Reference — Essential SQL Query Library

This section consolidates the most important day-to-day DBA queries into a single reference library. These queries cover all routine administration tasks and should be saved as a DBA toolkit for daily operations.

15.1 Daily Health Check Queries

SQL: Morning DBA Health Check Script

```
-- RUN THIS EVERY MORNING

-- 1. Active sessions count
SELECT COUNT(*) AS active_sessions
FROM sys.dm_exec_sessions WHERE is_user_process=1;

-- 2. Long-running queries (> 5 minutes)
SELECT session_id, login_name, status,
total_elapsed_time/60000.0 AS elapsed_min, wait_type
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id=s.session_id
WHERE r.total_elapsed_time > 300000 -- 5 minutes
AND r.session_id > 50;

-- 3. Failed queries last 24 hours
SELECT COUNT(*) AS failed_queries_24h
FROM queryinsights.exec_requests_history
WHERE status = 'Failed'
AND start_time >= DATEADD(HOUR, -24, GETUTCDATE());

-- 4. Any blocking?
SELECT COUNT(*) AS blocked_sessions
FROM sys.dm_exec_requests
WHERE blocking_session_id > 0;
```

SQL: Weekly Performance Summary Report

```
-- Weekly DBA performance report
SELECT
CAST(start_time AS DATE) AS query_date,
COUNT(*) AS total_queries,
SUM(CASE WHEN status='Succeeded'
THEN 1 ELSE 0 END) AS succeeded,
SUM(CASE WHEN status='Failed'
THEN 1 ELSE 0 END) AS failed,
SUM(CASE WHEN status='Cancelled'
THEN 1 ELSE 0 END) AS cancelled,
AVG(total_elapsed_time_ms)/1000.0 AS avg_duration_sec,
MAX(total_elapsed_time_ms)/1000.0 AS max_duration_sec,
COUNT(DISTINCT login_name) AS distinct_users
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY, -7, GETUTCDATE())
GROUP BY CAST(start_time AS DATE)
ORDER BY query_date;
```

15.2 Troubleshooting Quick Reference

SQL: Investigate a Specific Failed Query

```
-- Find details about a specific failed query
SELECT
distributed_statement_id,
login_name,
start_time,
end_time,
total_elapsed_time_ms/1000.0 AS elapsed_sec,
status,
row_count,
command
FROM queryinsights.exec_requests_history
WHERE status = 'Failed'
AND start_time >= DATEADD(HOUR, -2, GETUTCDATE())
ORDER BY start_time DESC;
```

SQL: Statistics Maintenance Script

```
-- Update statistics on all user tables (run after major loads)
DECLARE @tbl NVARCHAR(256);
DECLARE @sql NVARCHAR(512);
DECLARE cur CURSOR FOR
SELECT QUOTENAME(s.name)+'.'+QUOTENAME(t.name)
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE t.is_ms_shipped=0;

OPEN cur;
FETCH NEXT FROM cur INTO @tbl;
WHILE @@FETCH_STATUS = 0
BEGIN
SET @sql = 'UPDATE STATISTICS ' + @tbl;
EXEC sp_executesql @sql;
PRINT 'Updated stats: ' + @tbl;
FETCH NEXT FROM cur INTO @tbl;
END;
CLOSE cur; DEALLOCATE cur;
```

SQL: Full Permissions Report for All Objects

```
-- Export all grants for documentation and audit
SELECT
pr.name AS principal,
pr.type_desc AS principal_type,
pe.state_desc + ' ' + pe.permission_name AS permission,
pe.class_desc,
CASE pe.class
WHEN 0 THEN 'DATABASE'
WHEN 1 THEN OBJECT_NAME(pe.major_id)
WHEN 3 THEN SCHEMA_NAME(pe.major_id)
END AS secured_object
FROM sys.database_permissions pe
JOIN sys.database_principals pr
ON pe.grantee_principal_id = pr.principal_id
ORDER BY pr.name, pe.class_desc, pe.permission_name;
```

SQL: Database Object Count Summary

```
SELECT
'User Tables' AS category, COUNT(*) AS count
FROM sys.tables WHERE is_ms_shipped=0
UNION ALL SELECT 'Views', COUNT(*) FROM sys.views WHERE is_ms_shipped=0
UNION ALL SELECT 'Procedures',COUNT(*) FROM sys.procedures WHERE is_ms_shipped=0
UNION ALL SELECT 'Functions', COUNT(*) FROM sys.objects
WHERE type IN ('FN','IF','TF') AND is_ms_shipped=0
UNION ALL SELECT 'Statistics',COUNT(*) FROM sys.stats
WHERE auto_created=0 AND user_created=1
UNION ALL SELECT 'DB Users', COUNT(*) FROM sys.database_principals
WHERE type IN ('E','X','S') AND is_fixed_role=0;
```

15.3 Official Microsoft Learning Resources

Resource	URL	Purpose
Fabric Data Warehouse Docs	learn.microsoft.com/fabric/data-warehouse	Complete DW documentation
T-SQL Reference for Fabric	learn.microsoft.com/sql/t-sql/statements	T-SQL syntax reference
Fabric Security Overview	learn.microsoft.com/fabric/security	Security architecture
Performance Guidelines	learn.microsoft.com/fabric/data-warehouse/guidelines-warehouse-performance	Tuning best practices
Fabric Monitoring	learn.microsoft.com/fabric/data-warehouse/monitoring-overview	DMVs and Query Insights
BCDR Guidance	learn.microsoft.com/fabric/security/experience-specific-guidance	Disaster recovery
Git Integration	learn.microsoft.com/fabric/cicd/git-integration	Source control
DP-600 Certification	learn.microsoft.com/credentials/certifications/fabric-analytics-engineer-associate	Fabric certification
Fabric Blog	blog.fabric.microsoft.com	Latest features and updates
Fabric Community	community.fabric.microsoft.com	Q&A; and community support

IMPORTANT: All SQL queries in this learning material are written specifically for Microsoft Fabric Warehouse T-SQL syntax. Some queries use the queryinsights schema which is auto-provisioned in every Fabric Warehouse. Always test queries in a development environment before running in production.